



Unidade 2 — Frameworks



Roteiro da Aula (50 min)

Tempo	Atividade
0-5 min	Abertura e contextualização
5-15 min	Revisão dos conceitos da Unidade 2
15-25 min	Discussão: por que frameworks resolvem problemas reais
25-40 min	Demonstração prática com Express
40-47 min	Desafio: comparar Pull-based × Push-based
47-50 min	Síntese e orientação da atividade



0-5 min: Contextualização

O que estudamos na Unidade 2:

- Aula 07: O que é um Framework?
- Aula 08: Por que utilizar Frameworks?
- Aula 09: Estrutura de um Framework
- Aula 10: Frameworks Back-End e Front-End
- Aula 11: Pull-based × Push-based
- Aula 12: Como escolher um Framework?

O que vamos fazer hoje:

Construir uma aplicação com Express, demonstrando Router, Controller, Middleware, Service e Repository.



5-15 min: Revisão dos Conceitos

Componentes de um Framework:

Requisição → Router → Middleware → Controller → Service → Repository → Response

Paradigmas:

Pull-based	Push-based
Cliente puxa	Servidor empurra
HTTP / fetch	WebSockets / SSE
CRUD	Tempo real



15-25 min: Problema Real

Comparação: Manual vs. Framework

Aspecto	Manual (Unidade 1)	Com Framework (Unidade 2)
Servidor	<code>http.createServer()</code>	<code>express()</code>
Roteamento	<code>if/else</code>	<code>app.get()</code> , <code>app.post()</code>
Parse JSON	Manual com <code>req.on('data')</code>	<code>express.json()</code>
Arquivos estáticos	<code>fs.readFile()</code>	<code>express.static()</code>
Organização	Tudo em um arquivo	Pastas separadas
Middlewares	Implementar manualmente	<code>app.use()</code>



25-40 min: Demonstração Prática

Estrutura do projeto com Express:

```
contatos-framework/  
├── src/  
│   ├── routes/  
│   │   └── contatos.js  
│   ├── controllers/  
│   │   └── contatoController.js  
│   ├── services/  
│   │   └── contatoService.js  
│   └── repositories/  
│       └── contatoRepository.js  
├── public/  
│   ├── index.html  
│   ├── style.css  
│   └── app.js  
├── server.js  
└── package.json
```



Demonstração: Repository

```
// src/repositories/contatoRepository.js
let contatos = [];
let idCounter = 1;

const contatoRepository = {
  buscarTodos() {
    return contatos;
  },
  buscarPorId(id) {
    return contatos.find(c => c.id === id);
  },
  criar(dados) {
    const contato = { id: idCounter++, ...dados };
    contatos.push(contato);
    return contato;
  },
  atualizar(id, dados) {
    const index = contatos.findIndex(c => c.id === id);
    if (index === -1) return null;
    contatos[index] = { ...contatos[index], ...dados };
    return contatos[index];
  },
  excluir(id) {
    contatos = contatos.filter(c => c.id !== id);
  }
};

module.exports = contatoRepository;
```





Demonstração: Service

```
// src/services/contatoService.js
const contatoRepository = require('../repositories/contatoRepository');

const contatoService = {
  listar() {
    return contatoRepository.buscarTodos();
  },
  criar(dados) {
    if (!dados.nome || dados.nome.length < 2) {
      throw new Error('Nome deve ter pelo menos 2 caracteres');
    }
    if (!dados.email || !dados.email.includes('@')) {
      throw new Error('E-mail inválido');
    }
    return contatoRepository.criar(dados);
  },
  excluir(id) {
    const contato = contatoRepository.buscarPorId(id);
    if (!contato) throw new Error('Contato não encontrado');
    contatoRepository.excluir(id);
  }
};

module.exports = contatoService;
```





Demonstração: Controller

```
// src/controllers/contatoController.js
const contatoService = require('../services/contatoService');

const contatoController = {
  listar(req, res) {
    const contatos = contatoService.listar();
    res.json(contatos);
  },
  criar(req, res) {
    try {
      const contato = contatoService.criar(req.body);
      res.status(201).json(contato);
    } catch (error) {
      res.status(400).json({ erro: error.message });
    }
  },
  excluir(req, res) {
    try {
      contatoService.excluir(parseInt(req.params.id));
      res.status(204).send();
    } catch (error) {
      res.status(404).json({ erro: error.message });
    }
  }
};

module.exports = contatoController;
```





Demonstração: Routes

```
// src/routes/contatos.js
const express = require('express');
const router = express.Router();
const contatoController = require('../controllers/contatoController');

router.get('/', contatoController.listar);
router.post('/', contatoController.criar);
router.delete('/:id', contatoController.excluir);

module.exports = router;
```

```
// server.js
const express = require('express');
const app = express();
app.use(express.json());
app.use(express.static('public'));
app.use('/api/contatos', require('./src/routes/contatos'));
app.listen(3000, () => console.log('Servidor na porta 3000'));
```



40-47 min: Desafio

Desafio: Adicione a funcionalidade de **edição** e implemente um **middleware de log**.

Requisitos:

1. Criar rota `PUT /api/contatos/:id`;
2. Implementar no Controller, Service e Repository;
3. Criar um middleware que registre `MÉTODO URL - DATA` no console;
4. Aplicar o middleware a todas as rotas.



47-50 min: Síntese

Comparação final:

Aspecto	Manual	Framework
Linhas de infraestrutura	~100	~5
Organização	1 arquivo	Múltiplos arquivos
Roteamento	if/else	Declarativo
Middlewares	Manual	Integrado
Manutenção	Difícil	Fácil

Atividade Prática Orientada: disponível na pasta da Unidade 2.

